

Hangman – Proiect Devops

Pop Tudor

Descrierea cerinței

Implementarea unui pipeline CI/CD complet pentru o aplicație web

Scopul Proiectului: Proiectul are ca scop dezvoltarea și configurarea unui pipeline CI/CD (Continuous Integration/Continuous Deployment) pentru o aplicație web, utilizând unelte DevOps precum Jenkins, Docker și Kubernetes. Cursanții vor automatiza procesele de build, testare și deployment, asigurând un flux de livrare rapid și eficient.

Cerințe Generale:

1. Scenariul Proiectului:

- Dezvoltați și configurați un pipeline CI/CD pentru o aplicație web simplă (de exemplu, o aplicație de tip To-Do List sau o aplicație de tip portofoliu).
- Aplicația poate fi dezvoltată folosind orice limbaj de programare (Node.js, Python, Java, etc.) și trebuie să includă un minim de funcționalități de bază.

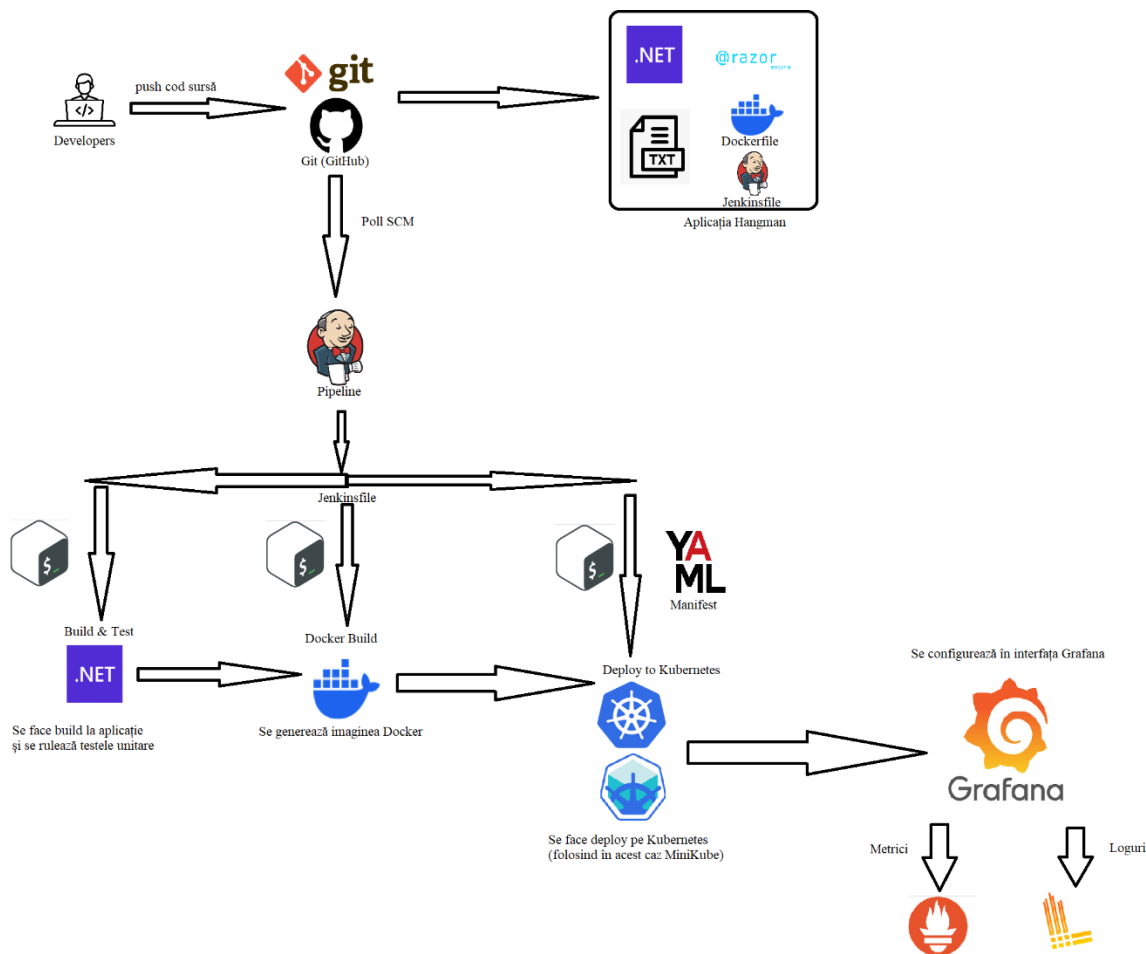
2. Elementele Proiectului:

- **Version Control și Repo:** Utilizați Git pentru controlul versiunilor și încărcăți codul sursă pe o platformă precum GitHub sau GitLab.
- **CI/CD Pipeline:**
 - **Configurarea Jenkins:** Configurați Jenkins pentru a declanșa pipeline-ul automat la fiecare commit în repo-ul Git.
 - **Build și Test:** Automatizați procesul de build al aplicației și implementați teste automate pentru validarea codului.
 - **Containerizare cu Docker:** Creați un Dockerfile pentru aplicație și configurați Jenkins pentru a genera imagini Docker.
 - **Deploy pe Kubernetes:** Configurați un cluster Kubernetes (local sau cloud) și implementați aplicația containerizată folosind Kubernetes.
- **Monitorizare și Logare:**
 - Implementați o soluție de monitorizare simplă (ex.: Prometheus și Grafana) pentru a urmări performanța aplicației și a infrastructurii.
 - Configurați logarea aplicației și colectarea logurilor (ex.: ELK Stack - Elasticsearch, Logstash, Kibana).

3. Livrarea Proiectului:

- Proiectul trebuie prezentat sub forma unui portofoliu care să includă codul sursă, fișierele de configurare ale pipeline-ului, documentația de instalare și utilizare, precum și capturi de ecran cu funcționarea pipeline-ului și monitorizarea.
- Toate configurațiile și codul sursă vor fi încărcate pe GitHub, GitLab sau Bitbucket, împreună cu un fișier README care să explice pașii de instalare și rulare.

Tehnologii folosite



C# - este un limbaj de programare de nivel înalt, de uz general, care susține multiple paradigme. C# înglobează discipline de programare cu tipizare statică, tipizare puternică, cu domeniu de vizibilitate lexical (lexical scoping), imperativă, declarativă, funcțională, generică, orientată pe obiecte (bazată pe clase) și orientată pe componente. Principalii designeri ai limbajului de programare C# au fost Anders Hejlsberg, Scott Wiltamuth și Peter Golde de la Microsoft. Acesta a fost distribuit pe scară largă pentru prima dată în iulie 2000 și a fost ulterior aprobat ca standard internațional de către Ecma (ECMA-334) în 2002 și de ISO/IEC (ISO/IEC 23270 și 20619) în 2003. Microsoft a introdus C# împreună cu .NET .

În proiectul Hangman, este limbajul de programare pe care rulează motorul jocului (HangmanEngine.cs) și testele unitare (EngineTests.cs).



.NET - este un software framework proprietar dezvoltat de Microsoft, care rulează în principal pe Microsoft Windows. A fost implementarea predominantă a Common Language Infrastructure (CLI) până când a fost înlocuită de proiectul cross-platform .NET. Acesta include o bibliotecă extinsă de clase numită Framework Class Library (FCL) și oferă interoperabilitate lingvistică (fiecare limbaj poate folosi cod scris în alte limbaje) între mai multe limbaje de programare. Programele scrise pentru .NET Framework rulează într-un mediu software (spre deosebire de un mediu hardware) numit Common Language Runtime (CLR). CLR este o mașină virtuală de aplicații care oferă servicii precum securitatea, gestionarea memoriei și gestionarea excepțiilor. Ca atare, codul informatic scris folosind .NET Framework este numit „cod gestionat” (managed code). FCL și CLR împreună constituie .NET Framework.



În aplicația Hangman constituie framework-ul de bază în care este scris jocul.

Visual Studio - este un mediu de dezvoltare integrat (IDE) dezvoltat de Microsoft. Este utilizat pentru a crea programe informatice, inclusiv site-uri web, aplicații web, servicii web și aplicații mobile. Visual Studio utilizează platformele de dezvoltare software Microsoft, precum Windows API, Windows Forms, Windows Presentation Foundation (WPF), Microsoft Store și Microsoft Silverlight. Acesta poate produce atât cod nativ, cât și cod gestionat (managed code). Visual Studio include un editor de cod care suportă IntelliSense (componenta de completare a codului), precum și refactorizarea codului. Depanatorul integrat (debugger) funcționează atât la nivel de cod sursă, cât și la nivel de mașină. Alte instrumente integrate includ un profiler de cod, un designer pentru construirea aplicațiilor cu interfață grafică (GUI), un designer web, un designer de clase și un designer de scheme de baze de date. Acesta acceptă module (plug-ins) care extind funcționalitatea la aproape orice nivel — incluzând adăugarea de suport pentru sisteme de control al versiunilor (cum ar fi Subversion și Git) și adăugarea de noi seturi de instrumente, cum ar fi editoare și designere vizuale pentru limbaje specifice unui domeniu sau seturi de instrumente pentru alte aspecte ale ciclului de viață al dezvoltării software.



Constituie mediul în care este scrisă aplicația Hangman.

HTML (Hypertext Markup Language) - este limbajul de marcare standard pentru documentele concepute pentru a fi afișate într-un browser web. Acesta definește conținutul și structura conținutului web. Este adesea asistat de tehnologii precum Cascading Style Sheets (CSS) și limbaje de scriptare precum JavaScript. Browsers web primesc documente HTML de la un server web sau din stocarea locală și redau (randează) documentele sub formă de pagini web multimedia. HTML descrie structura unei pagini web din punct de vedere semantic și,



inițial, includea și indicii referitoare la aspectul acesteia. A fost conceput de Sir Tim Berners-Lee, dezvoltat de WHATWG și World Wide Web Consortium (fost W3C) și lansat în 1993.

Razor - este o sintaxă de programare ASP.NET utilizată pentru a crea pagini web dinamice folosind limbajele de programare C# sau VB.NET. Razor a fost introdus în iunie 2010 și a fost lansat pentru Microsoft Visual Studio 2010 în ianuarie 2011. Razor este un motor de vizualizare (view engine) cu o sintaxă simplă și a fost lansat ca parte din MVC 3 și din setul de instrumente WebMatrix. Razor a devenit o componentă a ASP.NET WebStack, iar ulterior a devenit parte din ASP.NET Core. Sintaxa Razor este o sintaxă de marcare pentru șabloane (template markup syntax), bazată pe limbajul de programare C#, care îi permite programatorului să utilizeze un flux de lucru bazat pe construcția HTML. În loc să utilizeze sintaxa de marcare ASP.NET Web Forms (.aspx) cu simboluri `<%= %>` pentru a indica blocurile de cod, sintaxa Razor începe blocurile de cod cu caracterul `@` și nu necesită închiderea explicită a blocului de cod. Ideea din spatele Razor este de a oferi o sintaxă optimizată pentru generarea HTML folosind o abordare de șablonare axată pe cod, cu o tranziție minimă între HTML și cod. Designul reduce numărul de caractere și de apăsări de taste și permite un flux de lucru mai fluid prin faptul că nu necesită blocuri de server denotate explicit în interiorul codului HTML. Alte avantaje care au fost menționate:

- **Suportă IntelliSense** – suport pentru completarea instrucțiunilor.
- **Suportă „layout-uri”** – o alternativă la conceptul de „master page” din Web Forms clasic (.aspx).
- **Testabil la nivel de unitate** (Unit testable).

În contextul Hangman, Razor Pages este tehnologia din .NET responsabilă pentru interfața web (Index.cshtml). Ea randează paginile HTML în funcție de starea jocului transmisă de backend.

Fișier text (.txt) - este un tip de fișier de calculator structurat ca o secvență de linii de text electronic. Un fișier text este stocat ca date în cadrul sistemului de fișiere al unui calculator.

În contextul Hangman, este baza de date minimalistă a proiectului (words.txt). Fișierul conține lista de cuvinte din care aplicația extrage aleatoriu cuvântul care trebuie ghicit.



Git - este un sistem software de control al versiunilor distribuit, capabil să gestioneze versiuni ale codului sursă sau ale datelor. Este adesea utilizat pentru controlul codului sursă de către programatorii care dezvoltă software în mod colaborativ. A fost creat inițial de Linus Torvalds pentru controlul versiunilor în



dezvoltarea nucleului Linux. Obiectivele de design ale Git includ viteza, integritatea datelor și suportul pentru fluxuri de lucru distribuite și non-liniare — mii de ramuri (branch-uri) paralele care rulează pe computere diferite. La fel ca majoritatea celorlalte sisteme de control al versiunilor distribuite și spre deosebire de majoritatea sistemelor client-server, Git menține o copie locală a întregului depozit, cunoscut și sub numele de „repo”, cu posibilități de urmărire a istoricului și a versiunilor, independent de accesul la rețea sau de un server central. Un depozit este stocat pe fiecare computer într-un director standard, cu fișiere ascunse suplimentare pentru a oferi capacități de control al versiunilor. Git oferă funcționalități pentru a sincroniza modificările între depozitele care partajează un istoric; pentru colaborarea asincronă, acest lucru se extinde la depozitele de pe mașini aflate la distanță. Deși toate depozitele (cu același istoric) sunt egale, dezvoltatorii folosesc adesea un server central pentru a găzdui un depozit care să dețină o copie integrată.

GitHub - este o platformă proprietară pentru dezvoltatori care permite acestora să creeze, să stocheze, să gestioneze și să partajeze codul sursă. Aceasta utilizează Git pentru a oferi control distribuit al versiunilor, iar GitHub în sine oferă control al accesului, urmărire a erorilor (bug tracking), solicitări de funcționalități software, gestionarea sarcinilor, integrare continuă și pagini de tip wiki pentru fiecare proiect. GitHub, cu sediul central în San Francisco, este operat de GitHub, Inc., o filială a Microsoft din 2018.



În contextul Hangman, Git (reprezentat prin GitHub) este sistemul de versiuni unde se află codul. Orice push nou anunță serverul de automatizare că există modificări.

Bash (Bourne Again Shell) - este un interpretor de comenzi interactiv și un limbaj de comandă dezvoltat pentru sistemele de operare de tip Unix. Susținut de Free Software Foundation și creat în 1989 de Brian Fox pentru Proiectul GNU, acesta a fost conceput ca o alternativă software complet gratuită la shell-ul



Bourne (sh) și la alte shell-uri Unix proprietare. Beneficiind de o adopție pe scară largă, Bash este utilizat în mod obișnuit ca shell implicit de autentificare pentru numeroase distribuții Linux. De asemenea, acesta suportă execuția comenzilor din fișiere, cunoscute sub numele de scripturi shell (shell scripts), facilitând astfel automatizarea.

În contextul Hangman, este folosit pentru instalarea aplicațiilor necesare rulării și deployment-ului aplicației în mașina Unix, precum și pentru rularea comenzilor automate de testare, build, împachetare a imaginii Docker și deploy pe Kubernetes.

Docker - este un set de produse care utilizează virtualizarea la nivelul sistemului de operare pentru a livra software în pachete numite containere. Docker automatizează



implementarea aplicațiilor în interiorul unor containere ușoare, permițându-le acestora să ruleze în mod consecvent în medii de calcul diferite. Software-ul de bază care rulează și gestionează aceste containere se numește Docker Engine. Docker a fost lansat pentru prima dată în 2013 și continuă să fie dezvoltat de Docker, Inc. Platforma include atât versiuni gratuite, cât și versiuni cu plată. În contextul Hangman, folosește instrucțiunile din Dockerfile pentru a izola aplicația .NET și fișierul words.txt într-o imagine ușoară și portabilă, asigurându-se că aplicația va rula la fel peste tot.

Jenkins - este un server de automatizare open-source. Acesta ajută la automatizarea etapelor dezvoltării software legate de compilare (build), testare și implementare (deploy), facilitând integrarea continuă și livrarea continuă. Este un sistem bazat pe server care



Jenkins

rulează în containere de servlete precum Apache Tomcat sau, în mod implicit, ca o aplicație web autonomă în serverul Eclipse Jetty inclus. Acesta suportă instrumente de control al versiunilor, inclusiv AccuRev, CVS, Subversion, Git, Mercurial, Perforce, ClearCase și RTC, și poate executa proiecte bazate pe Apache Ant, Apache Maven și sbt, precum și scripturi shell arbitrare sau comenzi batch pentru Windows. A fost dezvoltat de Kohsuke Kawaguchi și lansat în 2011. Refeitor la Hangman, Jenkins este serverul de CI/CD care citește instrucțiunile din Jenkinsfile. Folosește scripturile Bash pentru a rula comenzi automate de testare, build, împachetare a imaginii Docker și deploy pe Kubernetes.

Apache Groovy - un limbaj de programare orientat pe obiecte, compatibil cu sintaxa Java, conceput pentru platforma Java. Este atât un limbaj static, cât și dinamic, având caracteristici similare cu cele din Python, Ruby și Smalltalk. Poate fi utilizat atât ca limbaj de programare, cât și ca limbaj de scripting pentru



Platforma Java, este compilat în bytecode pentru Java Virtual Machine (JVM) și interoperează perfect cu alte coduri și biblioteci Java. Groovy folosește o sintaxă cu acolade similară cu cea a Java. De asemenea, Groovy suportă closure-uri (închideri), șiruri de caractere pe mai multe linii și expresii incluse în interiorul șirurilor de caractere. Este limbajul în care este scris fișierul Jenkinsfile care descrie acțiunile pipeline-ului din Jenkins.

Kubernetes (K8s) - este un sistem open-source de orchestrare a containerelor pentru automatizarea implementării, scalării și gestionării software-ului. Conceput inițial de Google, proiectul este acum menținut de o comunitate globală de contributori, iar marca înregistrată este deținută de Cloud Native Computing Foundation. „Kubernetes” este adesea prescurtat prin contracția numerică „K8s”, care semnifică „litera K, urmată de 8 litere, urmată de s”. Kubernetes assemblează unul sau mai multe computere, fie mașini virtuale, fie servere fizice (bare metal), într-un cluster care poate rula sarcini de lucru (workloads) în containere. Acesta funcționează cu diverse medii de execuție pentru containere (container runtimes), cum ar fi containerd și CRI-O. Capacitatea sa de a rula și gestiona sarcini de lucru de toate dimensiunile și tipurile a dus la adoptarea sa pe scară largă în cloud și în centrele de date. Există multiple distribuții ale acestei platforme — de la furnizori independenți de software (ISV-uri), precum și oferte găzduite în cloud de la toți marii furnizori de servicii cloud publice. Software-ul constă dintr-un plan de control (control plane) și noduri pe care rulează aplicațiile propriu-zise. Include instrumente precum kubectl și kubernetes, care pot fi utilizate pentru a interacționa cu API-ul său bazat pe REST.



În contextul Hangman, Kubernetes este orchestratorul care preia imaginea Docker și o rulează în producție (sub formă de Pod-uri). Se asigură că aplicația are high-availability (scalare, restart automat în caz de crash).

Minikube - este un instrument software ușor și gratuit care creează un cluster Kubernetes cu un singur nod direct pe computerul tău local (macOS, Linux sau Windows). Este conceput special pentru a ajuta dezvoltatorii să testeze aplicații și să învețe Kubernetes fără a avea nevoie de resurse în cloud. Minikube rulează o mașină virtuală sau un container Docker și configurează un mediu Kubernetes simplificat în interiorul acestuia. Poți interacționa cu acest cluster folosind unealta nativă standard de linie de comandă, kubectl, permițându-ți să dezvolti, să rulezi și să depanezi manifestele Kubernetes local înainte de a le lansa într-un mediu de producție.



YAML - este un limbaj de serializare a datelor ușor de citit de către oameni. Este utilizat în mod obișnuit pentru fișierele de configurare și în aplicațiile în care datele sunt stocate sau transmise. YAML vizează multe dintre aceleași aplicații de comunicare ca și Extensible Markup Language (XML), dar are o sintaxă minimă care diferă în mod intenționat de Standard Generalized Markup Language (SGML). Acesta folosește indentarea în stil Python pentru a indica structura ierarhică (cuibărirea) și nu necesită ghilimele în jurul majorității valorilor de tip șir de caractere (suportă, de asemenea, stilul JSON cu [...] și {...} combinat în același fișier). Este limbajul în care este scris manifestul Kubernetes.



Helm - este un manager de pachete pentru Kubernetes. Acesta utilizează „chart-uri” ca format de pachet, bazat pe YAML. Oferă suport pentru definirea de șabloane folosind sintaxa de șabloane Go (Go template syntax), pentru a genera dinamic fișierele de configurare (manifeste) ale resurselor Kubernetes. În contextul Hangman, l-am folosit pentru a instala Grafana, Prometheus și Loki.



Grafana - este o aplicație web open-source pentru analiză și vizualizare. Aceasta se conectează la baze de date cu serii temporale și la alte surse de date, permițând utilizatorilor să construiască tablouri de bord (dashboards) care afișează metrice, jurnale (logs) și urme (traces). Grafana suportă surse de date precum Prometheus, AWS CloudWatch, Graphite, InfluxDB, Elasticsearch, PostgreSQL și MySQL. Torkel Odegaard a lansat Grafana în ianuarie 2014 ca urmare a activității sale la Graphite (software) în cadrul companiei Orbitz. Interfața cu utilizatorul a fost bazată inițial pe versiunea 3 a Kibana.



În contextul Hangman, Grafana este interfața grafică (Dashboard-ul) de monitorizare și logare. Grafana se conectează atât la Prometheus (pentru grafice de performanță), cât și la Loki (din versiunile recente, pentru vizualizarea logurilor), oferind un singur loc în care se poate vedea starea completă a întregului sistem.

Prometheus - este o aplicație software gratuită pentru monitorizarea evenimentelor și alertare. Acesta înregistrează metrice într-o bază de date cu serii temporale, construită utilizând un model de tip "HTTP pull" (extragere prin HTTP), care suportă o dimensionalitate ridicată prin perechi cheie-valoare pentru etichete, interogări flexibile și alertare în timp real. Proiectul este scris în limbajul Go și licențiat sub licența Apache 2.0, codul sursă fiind disponibil pe GitHub. Prometheus a luat naștere în cadrul SoundCloud în 2012 și a fost acceptat de Cloud Native Computing Foundation (CNCF) în 2016, finalizând etapa de incubare în 2018. Este utilizat frecvent împreună cu Grafana pentru vizualizarea tablourilor de bord și suportă o gamă largă de exportatoare (exporters) și integrări. Pentru Hangman, este sistemul de monitorizare care face „scraping” (colectează periodic date numerice/metrici) de la clusterul de Kubernetes și de la aplicația .NET (ex: utilizare CPU, RAM, număr de cereri HTTP, câte jocuri au fost jucate).

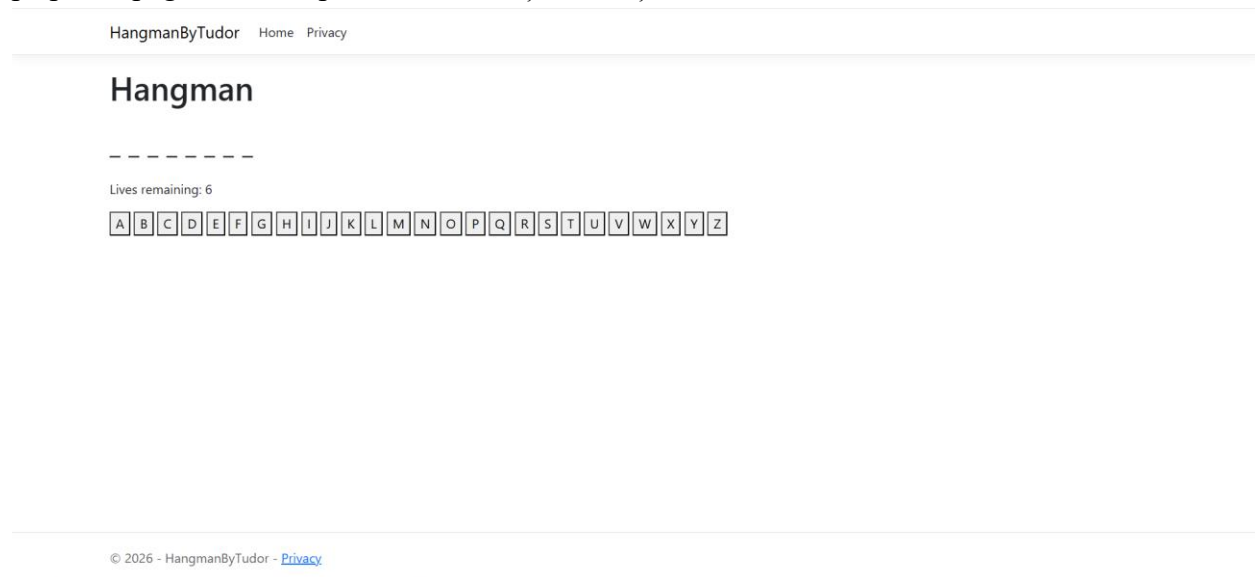


Grafana Loki - este un set de componente open-source care pot fi asamblate într-o platformă completă de gestionare a jurnalelor (logging stack). Un index de dimensiuni mici și segmente de date (chunks) puternic comprimate simplifică operarea și reduc semnificativ costurile asociate utilizării Loki. Spre deosebire de alte sisteme de logging, Loki este construit în jurul ideii de a indexa doar metadatele referitoare la etichetele (labels) jurnalelor din aplicație (la fel ca etichetele din Prometheus). Datele propriu-zise din jurnale sunt ulterior comprimate și stocate sub formă de segmente în sisteme de stocare de tip obiect (object stores), cum ar fi Amazon Simple Storage Service (S3), Google Cloud Storage (GCS) sau chiar local pe sistemul de fișiere. Referitor la Hangman, Loki este sistemul de colectare a logurilor. Acesta strânge în mod centralizat toate output-urile text generate de aplicația Hangman în timp ce rulează în Kubernetes (ex: erori de citire a fișierului, loguri de login, crash-uri).



Descrierea aplicației

Aplicația este un joc simplu de Hangman (spânzurătoare), realizat ca o aplicație web. Acesta apare pe prima pagină, necompletat, având inițial 6 vieți.



În caz de succes, utilizatorul va primi un mesaj că a câștigat.

Hangman

COMPILER

Lives remaining: 2

A B C D E F G H I J K L M N O P Q R S T U V W X Y Z

You Won!

[Play Again](#)

În caz de eșec, utilizatorul va primi mesajul Game Over. În ambele cazuri, utilizatorul poate începe un joc nou apăsând pe butonul Play Again.

Hangman

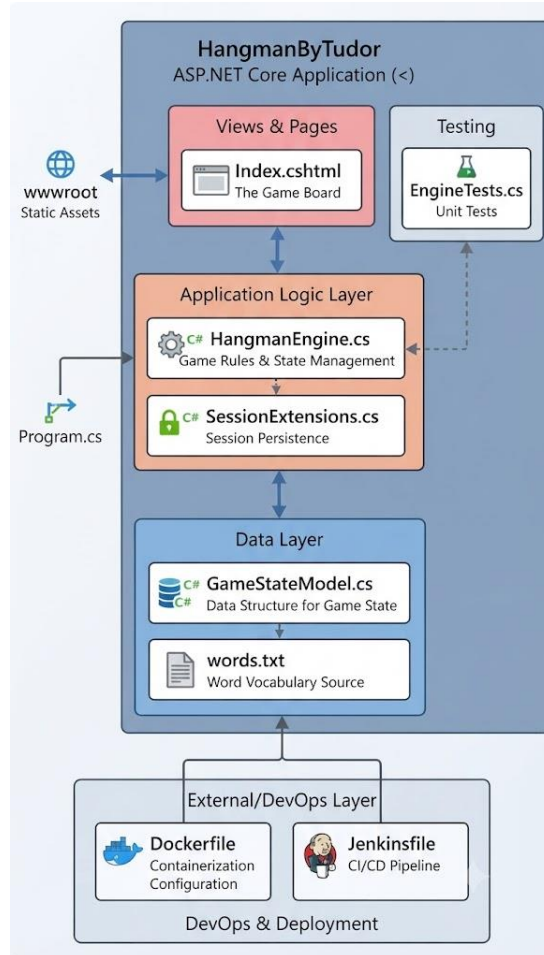
Lives remaining: 0

A B C D E F G H I J K L M N O P Q R S T U V W X Y Z

Game Over!

[Play Again](#)

Aplicația este realizată ca un proiect ASP.NET Core Web App (Razor Pages) în .NET 9.0 folosind Visual Studio 2022.



În fișierul `Program.cs` e scrisă clasa de bază care apelează metode din alte clase. Lista de cuvinte posibile este trecută în fișierul `words.txt`. În `SessionExtensions.js` sunt două metode, una care setează o valoare în sesiune pentru o anumită cheie și alta care citește acea valoare. Jocul ca obiect este definit în `GameStateModel.cs`, având ca proprietăți `SecretWord` de tip `string` (cu valoare implicită `string` gol / “”), o listă de caractere `GuessedLetters` (cu valoare implicită listă goală / `new()`), `Lives` de tip `int` (cu valoare implicită 6), `IsGameOver` de tip `bool` (cu valoare implicită dacă `Lives <= 0` sau dacă e câștigător / valoarea `IsWinner` e `true`) și `IsWinner`, tot de tip `bool` (cu valoarea implicită dată de faptul dacă toate literele din `SecretWord` se află în lista `GuessedLetters`). Algoritmul principal al jocului se află în fișierul `HangmanEngine.cs`. Aici se află metoda care procesează o încercare. Primind o literă ca parametru, dacă litera se află deja în `GuessedLetters`, sau jocul s-a terminat, nu se va modifica nimic. În caz diferit, litera se va adăuga în `GuessedLetters`, și dacă nu este conținută de `SecretWord`, se va scădea o viață din `Lives`. Pagina principală și restul logicii jocului sunt definite în fișierul `Razor Index.cshtml`. În mod obișnuit, un fișier cu acest nume va fi întotdeauna pagina Home (pagina implicită a aplicației). Ca orice fișier `Razor`, acesta poate conține atât cod `C#` cât și `HTML`. În `Index.cshtml`, se setează titlul paginii “Hangman Game”, se setează cuvântul de ghicit astfel, dacă litera a fost ghicită, se afișează litera, iar dacă nu, caracterul underscore (`_`). Ulterior se afișează viețile rămase, literele de la A la Z, dacă nu au fost alese, vor

fi selectabile și vor apărea cu negru. Dacă au fost alese, nu vor mai fi selectabile și vor apărea cu gri. Dacă jocul e câștigat, se va afișa “You Won!”, iar în caz contrar, “Game Over!”. La final este butonul “Play Again” care resetează formularul, astfel creând un obiect nou GameStateModel.

În sub-proiectul HangmanGame.Tests am implementat teste unitare cu scopul de a verifica corectitudinea codului scris. În EngineTests.cs am implementat două teste unitare, unul care verifică pentru un cuvânt dat, dacă, introducând o literă ce există în acel cuvânt, îmi menține numărul vieților și încă unul care, pentru același cuvânt, verifică dacă, introducând o literă ce nu există în acel cuvânt, îmi scade numărul vieților cu una.

În fișierul Dockerfile folosesc tehnica multi-stage build cu scopul de a obține o imagine finală cât mai mică și mai sigură. În prima fază îmi definesc imaginea de bază necesară doar pentru a rula aplicația, folosind aspnet:9.0, care conține doar bibliotecile necesare pentru rularea aplicațiilor ASP.NET Core, setând utilizatorul care rulează aplicația din motive de securitate (să nu ruleze ca root) și specificând porturile pe care aplicația va asculta (8080 pentru HTTP, 8081 pentru HTTPS). În a doua etapă am SDK-ul .NET necesar pentru a compila codul sursă. Cu dotnet restore îmi descarc toate pachetele NuGet necesare rulării proiectului, cu copy îmi copiez tot codul sursă în container și cu dotnet build compilez aplicația și plasez fișierele rezultate (binarele) în directorul /app/build. Această etapă ocupă mult spațiu, dar nu va fi inclusă în imaginea finală. Ulterior pregătesc fișierele pentru distribuție/producție, extrăgând fișierele compilate și pregătindu-le într-un format gata de rulat în folderul /app/publish. În cele din urmă, îmi creez imaginea folosită pentru rularea aplicației. Prin COPY --from=publish îmi copiez doar fișierele gata publicate din etapa anterioară (fișierele .dll și dependențele necesare). Acest lucru este necesar atât pentru eficientizare (o imagine Docker având SDK-ul complet pentru build putând ajunge și la 1GB), cât și pentru securitate (în cazul în care containerul ar fi compromis de un atacator, dacă imaginea ar conține și SDK-ul, ar putea folosi instrumentele instalate de compilare (cum ar fi dotnet build) pentru a compila și executa cod malițios în interiorul containerului). În ultimul pas, cu ENTRYPOINT definesc comanda care se execută automat la pornirea containerului pentru a lansa aplicația.

În fișierul Jenkinsfile îmi definesc un nou pipeline de CI/CD. Acesta reprezintă scenariul care îi spune server-ului Jenkins pașii preciși pe care trebuie să îi urmeze de fiecare dată când trimite cod nou în repository. La început am configurația de bază. Indic Jenkins-ului că poate rula acest pipeline pe orice nod (server) disponibil care are instrumentele necesare folosind agent any. Setez variabilele de mediu globale folosind environment. Aici se specifică calea către soluție (.sln) și se configurează unde se află instalat .NET pe server, pentru ca Jenkins să știe să execute comenzile dotnet. În stage-ul Build & Test validez codul. Cu build compilez proiectul pentru a vedea dacă am erori de sintaxă și cu test rulez testele automate. Folosind --no-build îi spun să nu mai compileze încă o dată pentru test și cu --logger generează un raport XML pe care Jenkins îl poate interpreta pentru a arăta grafice cu succesul testelor. În stage-ul Docker Build, Jenkins apelează Docker pentru a construi imaginea discutată anterior. În stage-ul Deploy to Kubernetes preiau imaginea Docker construită la pasul anterior și o forțez să ruleze în cluster. Întâi încarc imaginea în

MiniKube, după care aplic configurațiile Kubernetes, așa cum sunt trecute în fișierul manifest `k8s-deployment.yaml`. Actualizez imaginea în Deployment (`hangman-deployment`), schimbând varianta container-ului `hangman-app` cu noua versiune a imaginii, iar la final forțez epornirea (restart) tuturor pod-urilor (instanțelor aplicației) care aparțin de `hangman-deployment`, pentru aplicarea schimbărilor. La final am acțiunile post-execuție, `always` (indiferent dacă procesul a reușit sau a eșuat, Jenkins va procesa fișierul `test-results.xml` pentru a afișa rezultatele testelor direct în interfața web (Dashboard-ul Jenkins)) și `failure` (o comandă de alertă; dacă ceva nu merge bine, Jenkins va afișa acest mesaj în log-urile de eroare).

Fișierul manifest `k8s-deployment.yaml` definește resursele necesare pentru a rula aplicația `hangman` într-un cluster Kubernetes, cuprinzând un Deployment (pentru gestionarea containerelor) și un Service (pentru expunerea rețelei). Resursa Deployment asigură rularea instanțelor aplicației și gestionarea ciclului de viață al acestora. Specifică versiunea API-ului Kubernetes utilizată pentru resursele de tip Deployment prin valoarea `apiVersion`, `kind` este resursa care gestionează setul de pod-uri și actualizarea acestora, definește numărul de instanțe (pod-uri) care trebuie să ruleze simultan prin `replicas` și selector este criteriul prin care Deployment-ul identifică ce pod-uri trebuie să gestioneze. Ulterior configurez pod-ul (template) folosind următoarele valori: `labels`: eticheta (`app: hangman`) este aplicată pod-urilor create, permițând gruparea lor logică și `containers`, alcătuit din valorile: `name`: `hangman-app` (numele containerului în interiorul pod-ului), `image`: `hangman-game:latest` (imaginea Docker utilizată), `imagePullPolicy`: `IfNotPresent` - Kubernetes va încerca să utilizeze imaginea stocată local înainte de a încerca să o descarce din registru și `ports`: `containerPort`: 8080 - portul pe care aplicația ascultă în interiorul containerului. Ulterior, am resursa Service care oferă un punct de acces stabil pentru pod-urile aplicației. Având parametrii `kind` (abstracție care definește o cale de acces logică către un set de pod-uri), selector (prin care service-ul va trimite traficul către toate pod-urile care au eticheta `app: hangman`) și `type` (care expune service-ul pe un port static pe fiecare nod al clusterului, permițând accesul extern), resursa Service configurează port-urile (`port`: 80 fiind portul pe care service-ul îl ascultă pentru cererile primite din exteriorul clusterului iar `targetPort`: 8080 fiind portul către care este redirecționat traficul în cadrul containerului (se potrivește cu portul definit în Deployment)) și gestionează fluxul de trafic. Un utilizator accesează clusterul pe portul 80, `hangman-service` recepționează cererea, pe baza selectorului `app: hangman`, traficul este trimis către pod-ul definit de `hangman-deployment` și cererea ajunge la portul 8080 din interiorul containerului `hangman-app`.

Descrierea setărilor

În primul rând, este recomandată crearea proiectului în Git, pentru care am ales GitHub (altă variantă populară ar fi GitLab). După crearea proiectului, se clonează într-un director, după care se lucrează în acel director (notă: în imaginea de mai jos doar exemplific clonarea, aplicația fiind în acest punct deja creată).

```
TudorPop@DESKTOP-N8IC9HI MINGW64 /d/Hangman
$ git clone https://github.com/Teutorigos/HangmanByTudor.git
Cloning into 'HangmanByTudor'...
remote: Enumerating objects: 377, done.
remote: Counting objects: 100% (60/60), done.
remote: Compressing objects: 100% (42/42), done.
remote: Total 377 (delta 39), reused 38 (delta 18), pack-reused 317 (from 1)
Receiving objects: 100% (377/377), 1.02 MiB | 4.09 MiB/s, done.
Resolving deltas: 100% (220/220), done.
```

Din acest punct, orice modificare se va pune cu triada de comenzi git, add, commit, push. Cu git add . marcheaz toate fişierele modificate, pe care vreau să le trimit (folosind alte tool-uri precum Git Changes din Visual Studio, pot selecta mai uşor care dintre fişiere doresc să le trimit). Cu git commit, îmi salvez aceste fişiere pe plan local, astfel, inclusiv în cazul unor modificări nedorite, să pot ajunge uşor la versiunea salvată. Folosind git commit -m "nume_commit", pot da un nume acestui commit pentru a face mai facilă căutarea modificărilor în istoric. Ulterior, cu git push origin main trimit codul modificat în branch-ul main în acest caz.

```
TudorPop@DESKTOP-N8IC9HI MINGW64 ~/source/repos/Hangman/HangmanByTudor (main)
$ git add .

TudorPop@DESKTOP-N8IC9HI MINGW64 ~/source/repos/Hangman/HangmanByTudor (main)
$ git commit -m "Edited Jenkinsfile"
[main 63f8eea] Edited Jenkinsfile
1 file changed, 21 insertions(+)

TudorPop@DESKTOP-N8IC9HI MINGW64 ~/source/repos/Hangman/HangmanByTudor (main)
$ git push origin main
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 8 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (5/5), 652 bytes | 326.00 KiB/s, done.
Total 5 (delta 4), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (4/4), completed with 4 local objects.
To https://github.com/Teutorigos/HangmanByTudor.git
491baed..63f8eea main -> main
```

Aplicația poate fi văzută ulterior în serviciul de găzduire ales (GitHub în cazul meu).

← → ↻ github.com/Teutorigos/HangmanByTudor ☆

Platform Solutions Resources Open Source Enterprise Pricing 🔍 Sign in Sign up

Teutorigos / HangmanByTudor Public 🔔 Notifications 🍴 Fork 0 ☆ Star 0

<> Code Issues 🔄 Pull requests ⚙️ Actions 📁 Projects 🛡️ Security and quality 🔍 Insights

main 1 Branch 0 Tags 🔍 Go to file <> Code

Teutorigos Added logs 5feb744 · 2 days ago 57 Commits		
HangmanByTudor	Added logs	2 days ago
HangmanGame.Tests	Edited references	last month
.gitignore	Added application	last month
README.md	Initial commit	last month
k8s-deployment.yaml	Edited manifesto	4 days ago

README

HangmanByTudor

About

A simple Hangman game

- Readme
- Activity
- 0 stars
- 0 watching
- 0 forks

Report repository

Releases

No releases published

Packages

No packages published

Contributors 1

Ulterior, se instalează pe mașină dotnet, docker și jenkins, folosind instrucțiunile din fișierele .sh aferente:

instalareDotnet.sh:

```
Unpacking dotnet-targeting-pack-9.0 (9.0.17-0ubuntu1~24.04.1-ppa1) ...
Selecting previously unselected package dotnet-templates-9.0.
Preparing to unpack .../10-dotnet-templates-9.0_9.0.118-0ubuntu1~24.04.1-ppa1_amd64.deb ...
Unpacking dotnet-templates-9.0 (9.0.118-0ubuntu1~24.04.1-ppa1) ...
Selecting previously unselected package netstandard-targeting-pack-2.1-9.0.
Preparing to unpack .../11-netstandard-targeting-pack-2.1-9.0_9.0.118-0ubuntu1~24.04.1-ppa1_amd64.deb ...
Unpacking netstandard-targeting-pack-2.1-9.0 (9.0.118-0ubuntu1~24.04.1-ppa1) ...
Selecting previously unselected package dotnet-sdk-9.0.
Preparing to unpack .../12-dotnet-sdk-9.0_9.0.118-0ubuntu1~24.04.1-ppa1_amd64.deb ...
Unpacking dotnet-sdk-9.0 (9.0.118-0ubuntu1~24.04.1-ppa1) ...
Selecting previously unselected package dotnet-sdk-aot-9.0.
Preparing to unpack .../13-dotnet-sdk-aot-9.0_9.0.118-0ubuntu1~24.04.1-ppa1_amd64.deb ...
Unpacking dotnet-sdk-aot-9.0 (9.0.118-0ubuntu1~24.04.1-ppa1) ...
Setting up libltng-ust-common1t64:amd64 (2.13.7-1.1ubuntu2) ...
Setting up dotnet-targeting-pack-9.0 (9.0.17-0ubuntu1~24.04.1-ppa1) ...
Setting up dotnet-apphost-pack-9.0 (9.0.17-0ubuntu1~24.04.1-ppa1) ...
Setting up libltng-ust-ctl5t64:amd64 (2.13.7-1.1ubuntu2) ...
Setting up aspnetcore-targeting-pack-9.0 (9.0.17-0ubuntu1~24.04.1-ppa1) ...
Setting up dotnet-host-9.0 (9.0.17-0ubuntu1~24.04.1-ppa1) ...
Setting up netstandard-targeting-pack-2.1-9.0 (9.0.118-0ubuntu1~24.04.1-ppa1) ...
Setting up dotnet-hostfxr-9.0 (9.0.17-0ubuntu1~24.04.1-ppa1) ...
Setting up dotnet-templates-9.0 (9.0.118-0ubuntu1~24.04.1-ppa1) ...
Setting up libltng-ust1t64:amd64 (2.13.7-1.1ubuntu2) ...
Setting up dotnet-runtime-9.0 (9.0.17-0ubuntu1~24.04.1-ppa1) ...
Setting up aspnetcore-runtime-9.0 (9.0.17-0ubuntu1~24.04.1-ppa1) ...
Setting up dotnet-sdk-9.0 (9.0.118-0ubuntu1~24.04.1-ppa1) ...
Setting up dotnet-sdk-aot-9.0 (9.0.118-0ubuntu1~24.04.1-ppa1) ...
Processing triggers for man-db (2.12.0-4build2) ...
Processing triggers for libc-bin (2.39-0ubuntu8.7) ...
vboxuser@Linux2:~$ dotnet --version
9.0.315
```

instalareDocker.sh:


```

Setting up git (1:2.43.0-1ubuntu7.3) ...
Processing triggers for man-db (2.12.0-4build2) ...
vboxuser@LinuxTest1:~/Documents$ sudo docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
d5e71e642bf5: Download complete
Digest: sha256:96498fffd522e70807ab6384a5c0485a79b9c7c08ca79ba08623edcad1054e62d
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/

vboxuser@LinuxTest1:~/Documents$ sudo usermod -aG docker $USER
vboxuser@LinuxTest1:~/Documents$

```

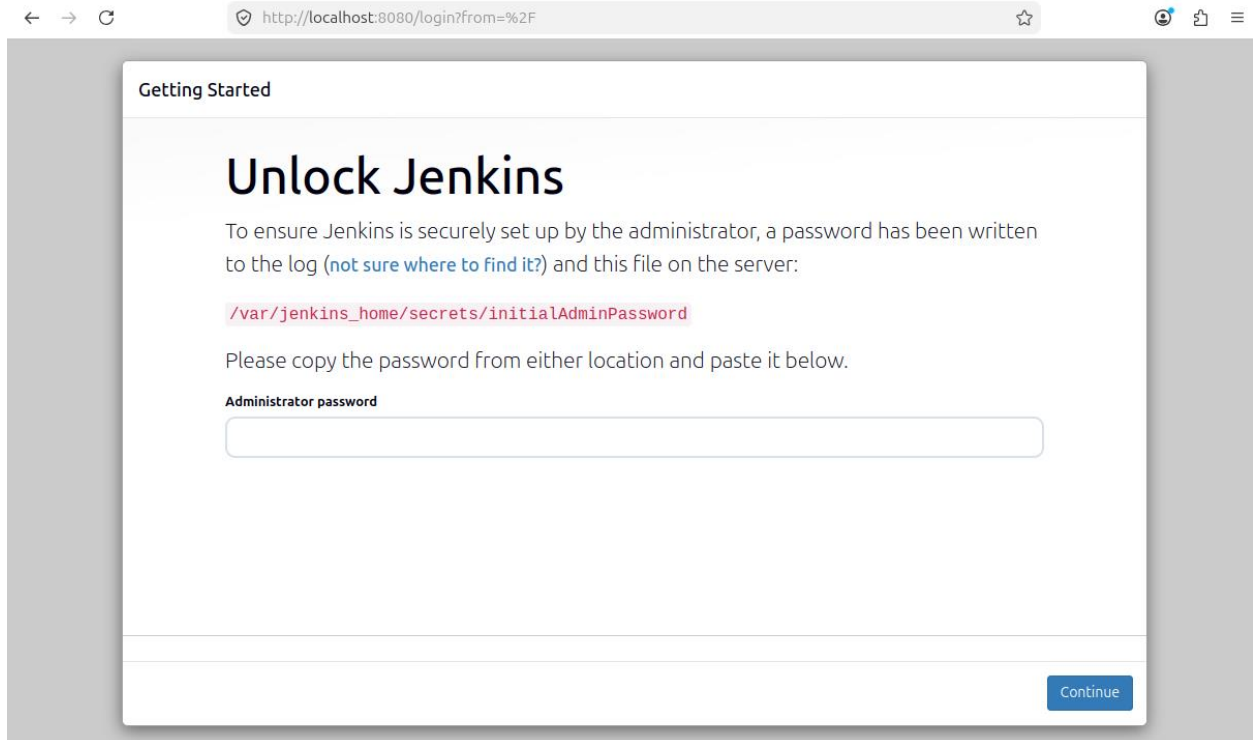
instalareJenkins.sh

```

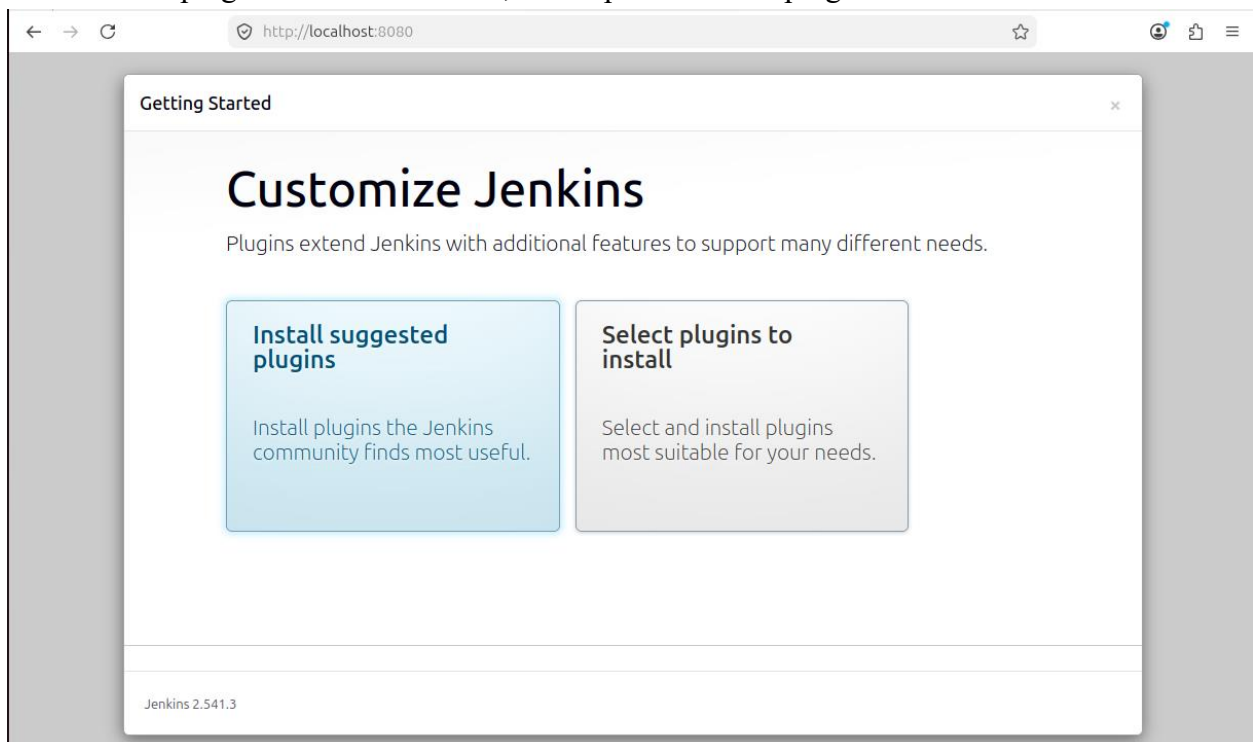
vboxuser@LinuxTest1:~/Documents$ sudo docker run -d -p 8080:8080 -p 50000:50000 -v jenkins_home:/var/jenkins_home --name
jenkins jenkins/jenkins:lts-jdk17
Unable to find image 'jenkins/jenkins:lts-jdk17' locally
lts-jdk17: Pulling from jenkins/jenkins
4d2125ccf71c: Pull complete
0b9e958129f4: Pull complete
69503609dc01: Pull complete
6da7907f70f4: Pull complete
6327a7366a6f: Pull complete
01938f7c040b: Pull complete
b7b154fd69a1: Pull complete
503dee4523cf: Pull complete
f44eb945cec1: Pull complete
46f93bf1cb2f: Pull complete
53c88f1dfef7: Pull complete
76eac8d3a597: Pull complete
c47cc6af1e21: Download complete
Digest: sha256:cce1eeb79902722f3cf7a8831c41e95c39f4a42b9a2981959630cc66539b3c2d
Status: Downloaded newer image for jenkins/jenkins:lts-jdk17
47f25a596db9a96f99a96c1760f7511d5eeb90614334aa034bb80a695dd645f9
vboxuser@LinuxTest1:~/Documents$ sudo docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword
32197b4cd4d5468fb406d03c3cc553f3
vboxuser@LinuxTest1:~/Documents$

```

După instalarea Jenkins, trebuie intrat pe <http://localhost:8080> și setată parola de administrator vizibilă cu: `sudo docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword`



Se instalează plugin-urile recomandate, sau se pot selecta ce plugin-uri să se instaleze.



Acum se poate crea pipeline-ul pentru aplicația Hangman. La configurări am pus * * * * * la Poll SCM pentru a interoga în fiecare minut dacă au apărut schimbări, astfel rulându-se build-ul la

fiicare

modificare

a

codului

← → ↻ Not Secure http://10.0.2.15:8080/job/HangmanPipeline/configure

Jenkins / HangmanPipeline / Configure

Configure

- General
- Triggers**
- Pipeline
- Advanced

Triggers

Set up automated actions that start your build based on specific events, like code changes or scheduled times.

- ☐ Build after other projects are built ?
- ☐ Build periodically ?
- ☐ GitHub hook trigger for GITScm polling ?
- ☒ Poll SCM ?

Schedule ?

⚠ Do you really mean "every minute" when you say "*****"? Perhaps you meant "H ****" to poll once per hour
Would last have run at Sunday, July 12, 2026, 9:35:00 AM Coordinated Universal Time; would next run at Sunday, July 12, 2026, 9:36:00 AM Coordinated Universal Time.

- ☐ Ignore post-commit hooks ?
- ☐ Trigger builds remotely (e.g., from scripts) ?

Save Apply

și am setat repository-ul din Git care să fie folosit

← → ↻ Not Secure http://10.0.2.15:8080/job/HangmanPipeline/configure

Jenkins / HangmanPipeline / Configure

Configure

- General
- Triggers
- Pipeline**
- Advanced

Pipeline script from SCM

SCM ?

Git

Repositories ?

Repository URL ?

https://github.com/Teutorigos/HangmanByTudor.git

Credentials ?

- none - + Add

Advanced ▾

+ Add Repository

Save Apply

precum și path-ul fișierului Jenkinsfile

← → ↻ Not Secure http://10.0.2.15:8080/job/HangmanPipeline/configure

Jenkins / HangmanPipeline / Configure

Configure

- General
- Triggers
- Pipeline
- Advanced**

Additional Behaviours

+ Add

Script Path ?

HangmanByTudor/Jenkinsfile

☒ Lightweight checkout ?

[Pipeline Syntax](#)

Advanced

Advanced ▾

Save Apply

REST API Jenkins 2.555.2

Ulterior, se face build.

← → ↻ Not Secure http://10.0.2.15:8080/job/HangmanPipeline/82/

Jenkins / HangmanPipeline / #82

Status

</> Changes

Console Output

Edit Build Information

Delete build '#82'

Polling Log

Timings

Git Build Data

Tests

Pipeline Overview

Restart from Stage

Replay

Pipeline Steps

Workspaces

Previous Build

✓ #82 (Jul 10, 2026, 9:14:11 AM)

Add description Keep this build forever

Started by an SCM change

Started 2 days 0 hr ago
Took 2 min 20 sec

This run spent:

- 9.6 sec waiting;
- 2 min 20 sec build duration;
- 2 min 29 sec total from scheduled to completion.

Revision: 63f8eeac4d60ed015f5ac316958df8b1a3d544c3
Repository: <https://github.com/Teutorigos/HangmanByTudor.git>

- refs/remotes/origin/main

Tests (no failures)

Edited Jenkinsfile 63f8eea
| popint94 at 9:13 AM 7/10/26

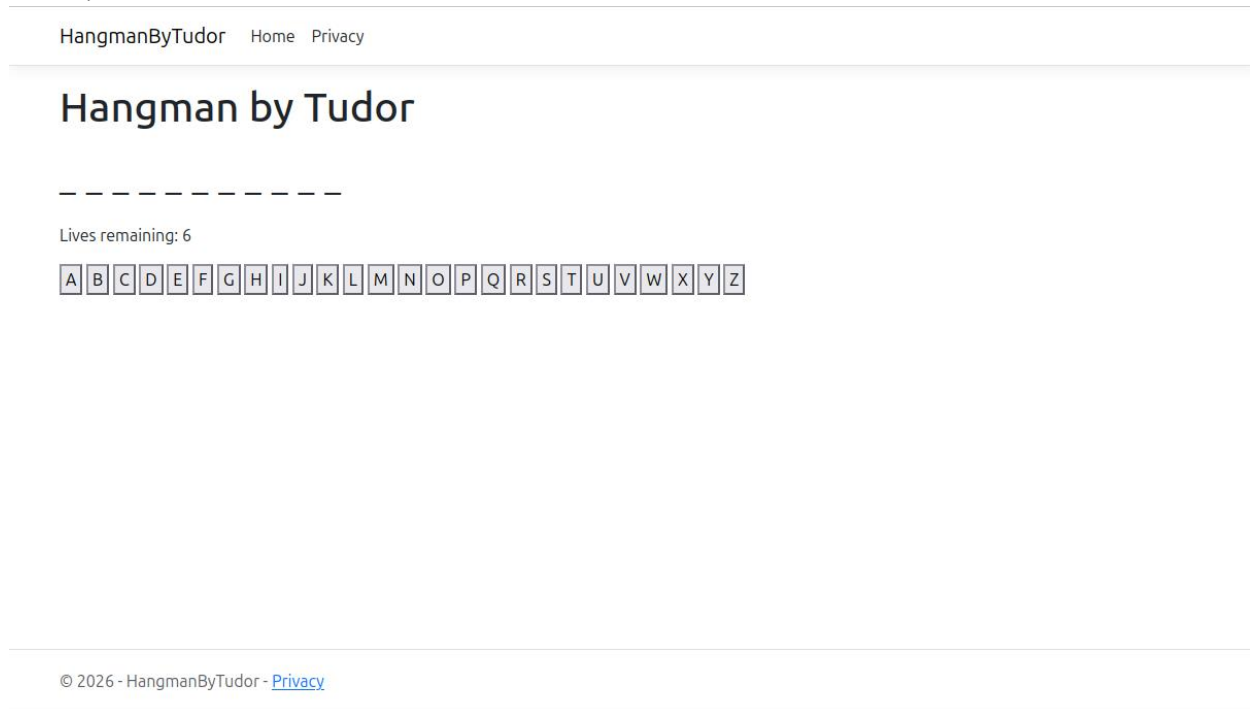
Următorul pas este instalarea MiniKube-ului.

```
vboxuser@LinuxTest1:~/Documents$ sudo curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total     Spent    Left     Speed
100 128M  100 128M    0     0  9842k      0  0:00:13  0:00:13 --:--:-- 10.9M
vboxuser@LinuxTest1:~/Documents$ sudo install minikube-linux-amd64 /usr/local/bin/minikube
vboxuser@LinuxTest1:~/Documents$
```

După instalare, i-am acordat permisiuni user-ului Jenkins ca să se poată modifica automat aplicația în urma rulării pipeline-ului din Jenkins și am pornit cluster-ul MiniKube sub user-ul Jenkins.

```
vboxuser@Linux2:~$ sudo -u jenkins minikube start -p minikube
🐳 minikube v1.38.1 on Ubuntu 24.04 (vbox/amd64)
🌟 Using the docker driver based on existing profile
👍 Starting "minikube" primary control-plane node in "minikube" cluster
🔄 Pulling base image v0.0.50 ...
🔧 Updating the running docker "minikube" container ...
🔧 Preparing Kubernetes v1.35.1 on Docker 29.2.1 ...
🔍 Verifying Kubernetes components...
   ▪ Using image gcr.io/k8s-minikube/storage-provisioner:v5
🌟 Enabled addons: storage-provisioner, default-storageclass
🎉 Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
```

Rulând pipeline-ul, cu instrucțiunile din Jenkinsfile, discutate mai sus, am reușit să fac deploy aplicației.



Ultimul pas este monitorizarea aplicației. Am instalat helm pentru a putea pune ulterior Grafana, Prometheus și Loki.

```
vboxuser@Linux2:~$ curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
% Total % Received % Xferd Average Speed Time Time Time Current
Dload Upload Total Spent Left Speed
100 11929 100 11929 0 0 91112 0 --:--:-- --:--:-- --:--:-- 91761
Downloading https://get.helm.sh/helm-v3.21.2-linux-amd64.tar.gz
Verifying checksum... Done.
Preparing to install helm into /usr/local/bin
[sudo] password for vboxuser:
helm installed into /usr/local/bin/helm
```

După aceea am instalat Prometheus, Loki și Grafana.

```
vboxuser@Linux2:~$ sudo -u jenkins helm install prometheus prometheus-community/prometheus \
--namespace monitoring \
--set alertmanager.enabled=false \
--set server.resources.requests.memory=256Mi \
--set server.resources.limits.memory=512Mi
NAME: prometheus
LAST DEPLOYED: Thu Jul 9 07:51:08 2026
NAMESPACE: monitoring
STATUS: deployed
REVISION: 1
TEST SUITE: None
NOTES:
The Prometheus server can be accessed via port 80 on the following DNS name from within your cluster:
prometheus-server.monitoring.svc.cluster.local

Get the Prometheus server URL by running these commands in the same shell:
export POD_NAME=$(kubectl get pods --namespace monitoring -l "app.kubernetes.io/name=prometheus,app.kubernetes.io/instance=prometheus" -o jsonpath="{.items[0].metadata.name}")
kubectl --namespace monitoring port-forward $POD_NAME 9090

Prometheus Pushgateway can be accessed via port 9091 on the following DNS name from within your cluster:
prometheus-prometheus-pushgateway.monitoring.svc.cluster.local

Get the Pushgateway URL by running these commands in the same shell:
export POD_NAME=$(kubectl get pods --namespace monitoring -l "app.kubernetes.io/name=prometheus-pushgateway,app.kubernetes.io/instance=prometheus" -o jsonpath="{.items[0].metadata.name}")
kubectl --namespace monitoring port-forward $POD_NAME 9091

vboxuser@Linux2:~$ sudo -u jenkins helm install loki grafana/loki-stack \
--namespace monitoring \
--set loki.persistence.enabled=false \
--set promtail.enabled=false \
--set fluent-bit.enabled=false \
--set grafana.enabled=false \
--set prometheus.enabled=false
WARNING: This chart is deprecated
NAME: loki
LAST DEPLOYED: Thu Jul 9 08:04:59 2026
NAMESPACE: monitoring
STATUS: deployed
REVISION: 1
NOTES:
The Loki stack has been deployed to your cluster. Loki can now be added as a datasource in Grafana.
See http://docs.grafana.org/features/datasources/loki/ for more detail.
```

```
vboxuser@Linux2:~$ sudo -u jenkins helm install grafana grafana/grafana \
--namespace monitoring \
--set persistence.enabled=false
WARNING: This chart is deprecated
NAME: grafana
LAST DEPLOYED: Thu Jul 9 07:51:54 2026
NAMESPACE: monitoring
STATUS: deployed
REVISION: 1
NOTES:
1. Get your 'admin' user password by running:

    kubectl get secret --namespace monitoring grafana -o jsonpath="{.data.admin-password}" | base64 --decode ; echo

2. The Grafana server can be accessed via port 80 on the following DNS name from within your cluster:

    grafana.monitoring.svc.cluster.local

    Get the Grafana URL to visit by running these commands in the same shell:
    export POD_NAME=$(kubectl get pods --namespace monitoring -l "app.kubernetes.io/name=grafana,app.kubernetes.io/instance=grafana" -o jsonpath="{.items[0].metadata.name}")
    kubectl --namespace monitoring port-forward $POD_NAME 3000

3. Login with the password from step 1 and the username: admin
#####
##### WARNING: Persistence is disabled!!! You will lose your data when #####
##### the Grafana pod is terminated. #####
#####
```

Ulterior acestor pași, se obține o parolă de administrator pentru Grafana folosind comanda

```
sudo -u jenkins minikube kubectl -- get secret --namespace monitoring grafana -o jsonpath="{.data.admin-password}" | base64 --decode ; echo
```

```
vboxuser@Linux2:~$ sudo -u jenkins minikube kubectl -- get secret --namespace monitoring grafana -o jsonpath="{.data.admin-password}" | base64 --decode ; echo
aoBD25GYeYrc82r8dzlK5qC6YNzAP0HuMjenHWY3
```

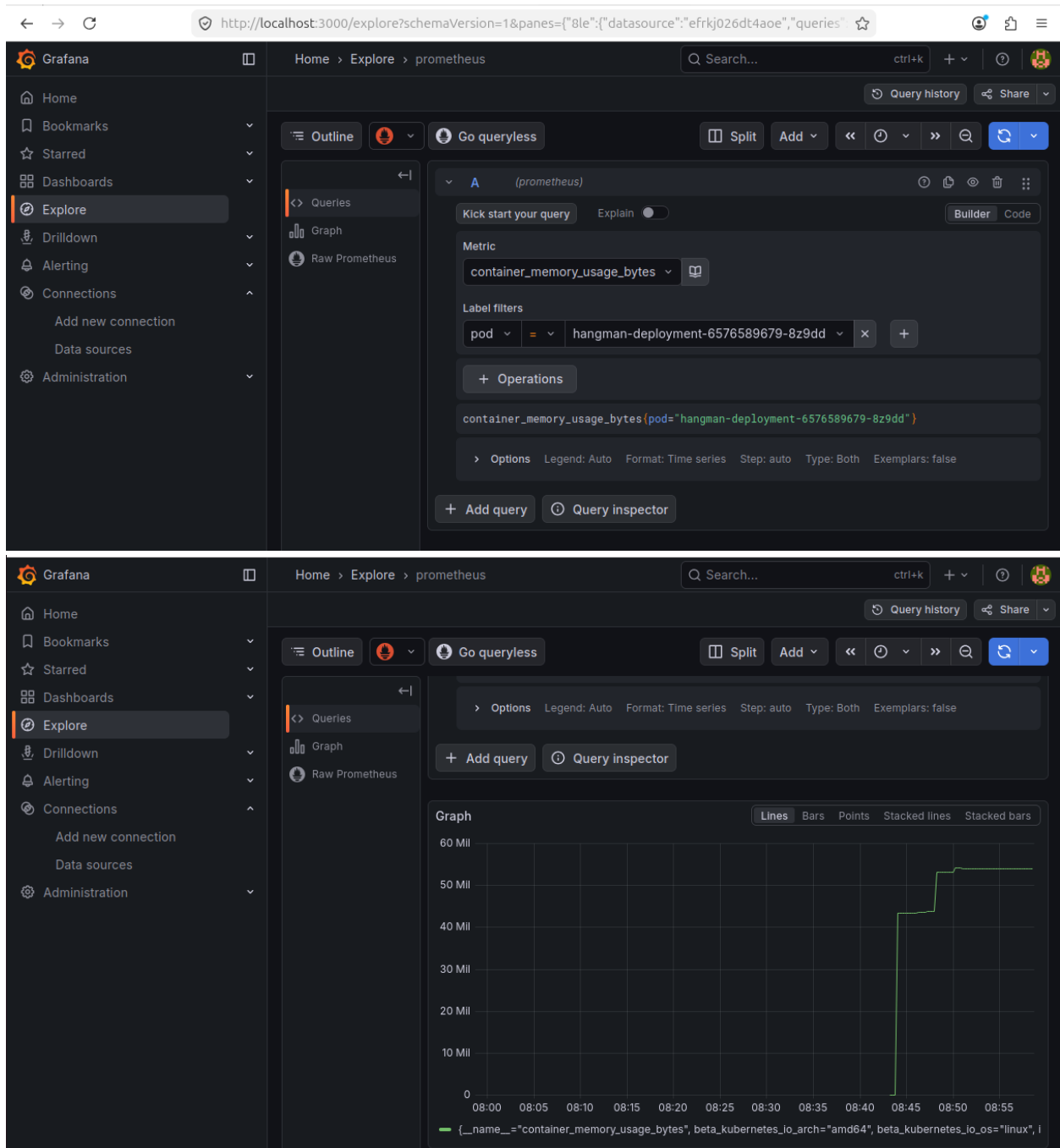
și, folosind username admin și această parolă, se poate porni Grafana.

```
vboxuser@Linux2:~$ sudo -u jenkins minikube kubectl -- port-forward --namespace monitoring service/grafana 3000:80
Forwarding from 127.0.0.1:3000 -> 3000
Forwarding from [::1]:3000 -> 3000
Handling connection for 3000
Handling connection for 3000
Handling connection for 3000
Handling connection for 3000
Handling connection for 3000
Handling connection for 3000
```

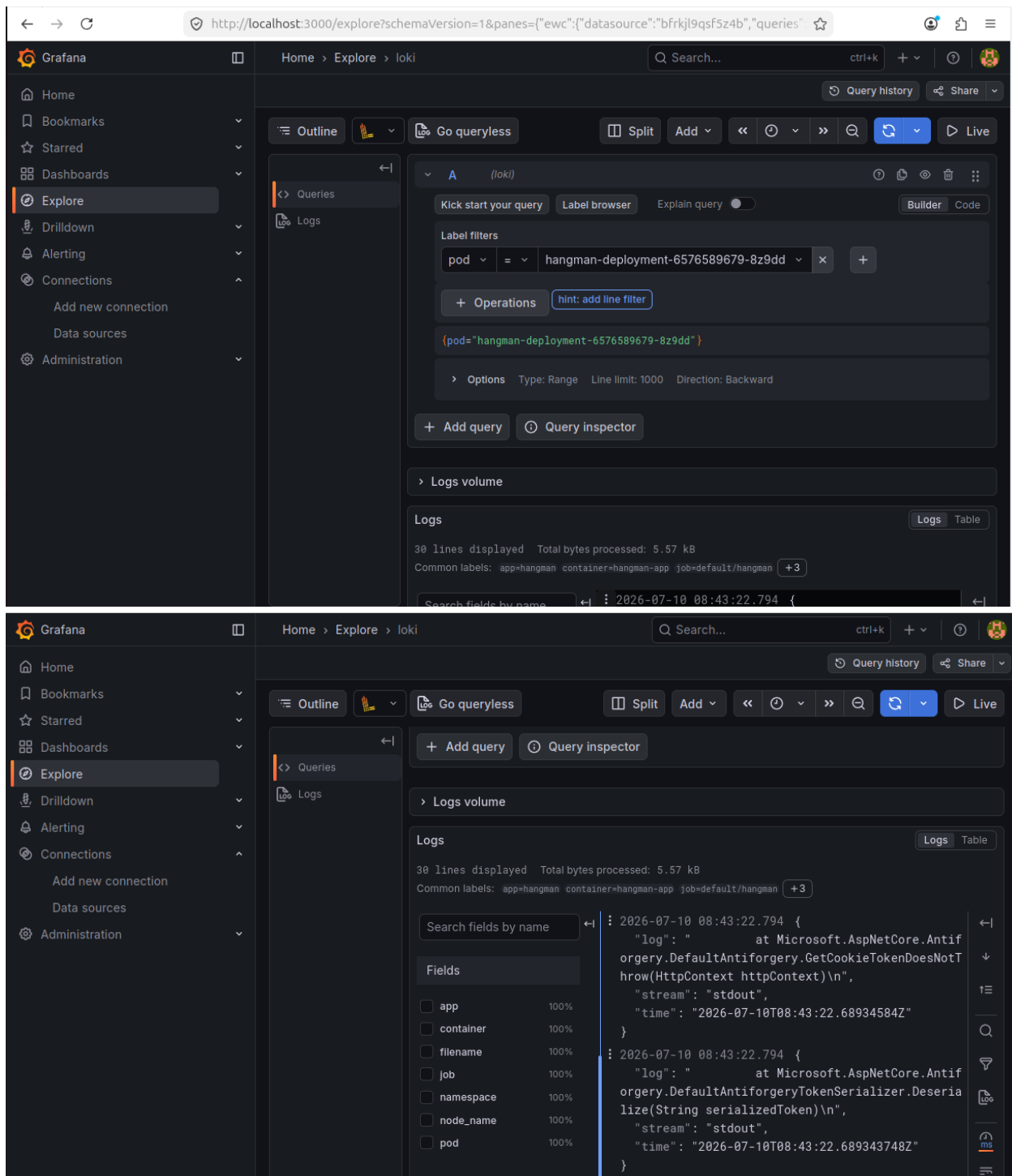
În interiorul Grafana, pentru Prometheus, am selectat să se caute după pod și am obținut pod-ul activ din Minikube folosind comanda

```
sudo -u jenkins minikube kubectl -- get pods
```

```
vboxuser@Linux2:~$ sudo -u jenkins minikube kubectl -- get pods
NAME                                READY   STATUS    RESTARTS   AGE
hangman-deployment-6576589679-8z9dd 1/1     Running   0           3m49s
vboxuser@Linux2:~$
```



și la fel și pentru Loki



Concluzie

Acest document își propune să explice atât tehnologiile folosite pentru crearea proiectului Hangman, cât și codul și setările necesare pentru realizarea unui flux complet de CI/CD, folosind Jenkins, Docker, Kubernetes, Grafana, Prometheus și Loki.